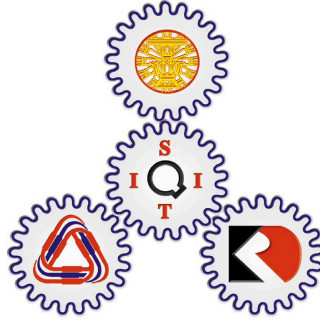




Database System Design

Kittin Traisiwakul	6622780409
Thinnaphop Direkpok	6622781969
Jetnipat Wanichsakul	6622780318
Kanade Areepoonsiri	6622770442
Patipat Permsub	6622780151
Pisit Pongpan	6622781852

CSS325: Database System

Asst. Prof. Dr. Preecha Tangworakitthaworn

SIRINDHORN INTERNATIONAL INSTITUTE OF TECHNOLOGY

THAMMASAT UNIVERSITY

November, 2025

Table of Contents

Reason why we have 4 entities	3
Revised Mini ERD and Revised Relation Schema	4
Pre-Processing	6
Data Dictionary	16
Create a database in MAMP server (DDL)	38
Example of Data Manipulation Language (DML)	46
Random data record using Python	53
References	80

Reason why we have 4 entities and 1 relationship (M:N) from Kaggle

First submission: 30 Aug - selecting data from kaggle

First, we selected this Kaggle file because it consists of 7 (.csv) files (aisle, departments, order_products_prior, order_products_train, orders, products, and sample_submission). The topic itself is interesting, and we thought that it qualified the criteria of having 3 master files and 2 transaction files.

After the first submission, we got feedback from the professor that we chose a “Good dataset”, and by that time, we still lacked the knowledge to be able to point out the flaws in the Kaggle dataset. As per that time, we still have not learned about how to draw relationship-schema yet, which includes Primary Key and Foreign Key. Furthermore, the erd/erd we practiced at that time is still a basic concept of knowing the relationship between entities, but not yet to be implemented on a big scale.

Second submission: 20 Sep - drawing erd/erd

By the time we do the second submission, we start writing a blueprint for our business project by drawing erd/erd diagram. The erd/erd diagram is being designed by ourselves according to the business rule we determined.

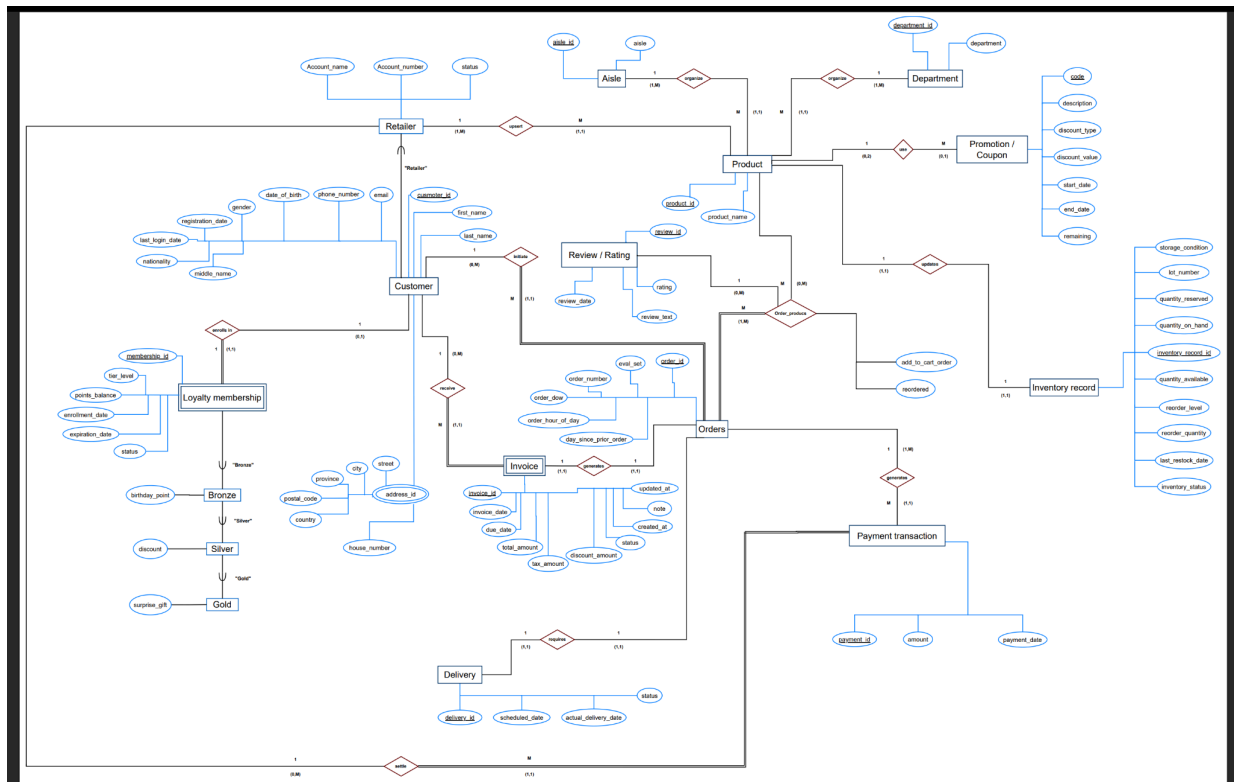
According to the “Good dataset” feedback, we all thought that our data is good to go and they are all entities. Hence, we designed the erd/erd diagram while considering the 5 files from Kaggle as entities, so as the relationship between each of them.

Third submission: 18 October - drawn schema, didn't check the kaggle's data

In our third submission, we are now implementing the 8 steps relational-schema to our project. However, as we have the idea that the 5 Kaggle files we chose from phase 1 are all entities. We do the relational-schema according to the mini erd diagram we designed, which considered that the 5 Kaggle files are all entities.

This results in us to not go back and check the original Kaggle file until we start doing the *fourth submission: 1 November*, which we need to develop the data dictionary based on our relational database schema. In doing this fourth submission, we have to load and insert data from the Kaggle dataset that is selected from phase 1. For this reason, we go back and check the 5 Kaggle files thoroughly and found out that one of the 5 Kaggle files we chose is an associated entity, which resulted from an many-to-many relationship between 2 Kaggle entities (files).

Revised Mini ERD and Revised Relation Schema





Pre-Processing

The 3 files from Kaggle are too large to be implemented in our project, so we thought of a pre-processing method to scale down the files to only implement instances that are necessarily enough for each table. First off, we looked into all these three files: order, order_product (prior&train), and product; after a close inspection, we can scale down the file by relying on order table because it have two meaningful attributes (“order_hour_of_day” [0-23, representing the hour of the day] and “order_dow” [0-6, representing the date of the week]) that is fit to be used as a criteria for scaling down all the three files.

Remark:

Code needs to be run according to this order.

The code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

1. **order_filter1:**

Initially, we have about 3.5 million rows of instances in order tables from Kaggle. We then scaled it down to 1,000 rows of instances by depending on the two instances (“order_hour_of_day” and “order_dow”). First off, we set a grouping function for dividing the data according to both attributes: “order_dow” - 7 groups (1 group per day) and “order_hour_of_day” - 4 groups (each group has a range of 6 hours). We then do a permutation (pairing from the two groupings to be distinct groups): $7 \times 4 = 28$ groups.

Now, we randomly sampled data into the 28 groups: $1000/28 = 35$.

Code: `sampled = ( df.groupby([dow, hour_bin], group_keys=False) .apply(lambda x: x.sample(n=min(len(x), n_per_group), random_state=42)) )`

This code means that every instances are being implemented into each group according to a dataframe (“order_hour_of_day” and “order_dow”) and it also ensures that the instances that are sampled into the group will not exceed 35 rows, as that every group will have the same number of rows.

We then also random “user_id” for 200 customers from initially 1,000 instances. This is because 200 instances is sufficient enough for the table. Then we changed the column name from “user_id” to “customer_id”.

The end result will generate a new file called “orders_sampled”.

Code for order_filter1:

```
import pandas as pd
import numpy as np

# Load the CSV
df = pd.read_csv("D:/SIIT/Project_DB_Lecture/data/orders.csv")

# Define bins for order_hour_of_day
bins = [0, 6, 12, 18, 24]
labels = [0, 1, 2, 3] # temporary codes for 4 parts

# Temporary grouping
hour_bin = pd.cut(df["order_hour_of_day"], bins=bins, labels=labels, right=False)
dow = df["order_dow"]

# Number of groups and per-group sample size
n_total = 1000
n_groups = 7 * 4 # 28
n_per_group = n_total // n_groups # ~35

# Stratified sampling
sampled = (
    df.groupby([dow, hour_bin], group_keys=False)
    .apply(lambda x: x.sample(n=min(len(x), n_per_group), random_state=42))
)

# Reassign user_id randomly in range 1–200
sampled["user_id"] = np.random.randint(1, 201, size=len(sampled))

# Rename user_id → customer_id
sampled = sampled.rename(columns={"user_id": "customer_id"})
```

```
# Save only the original columns (with renamed column)
sampled = sampled.rename(columns={"user_id":
"customer_id"})[df.columns.str.replace("user_id", "customer_id")]
sampled.to_csv("orders_sampled.csv", index=False)

print(f'Created 'orders_sampled.csv' with {len(sampled)} rows, "
      f'customer_id reassigned randomly between 1 and 200.')

```

2. **order_product_filter2:**

We first concatenate "order_product_prior" and "order_product_train" into one file, "order_product_all". Then we will sample the order-product data by comparing and mapping the "order_id" from "order_product_all" to the "order_id" in "orders_sampled," saving the matched records as "order_products_filtered." Similarly, we will match the "order_id" from "orders_sampled" to those in "order_product_all" and save the resulting data as "orders_sampled_filtered."

Code for order_product_filter2:

```
import pandas as pd

# Load files
orders_sampled = pd.read_csv("D:/SIIT/Project_DB_Lecture/orders_sampled.csv")
op_prior = pd.read_csv("D:/SIIT/Project_DB_Lecture/data/order_products__prior.csv")
op_train = pd.read_csv("D:/SIIT/Project_DB_Lecture/data/order_products__train.csv")

# Step 1: combine prior + train
order_products_all = pd.concat([op_prior, op_train], ignore_index=True)

# Step 2: filter combined order_products by sampled order_ids
order_products_filtered = order_products_all[
    order_products_all["order_id"].isin(orders_sampled["order_id"])
]

# Step 3: filter orders_sampled to keep only those present in combined file
orders_sampled_filtered = orders_sampled[
    orders_sampled["order_id"].isin(order_products_filtered["order_id"])
]

```



```
# Save results
order_products_filtered.to_csv("order_products_filtered.csv", index=False)
orders_sampled_filtered.to_csv("orders_sampled_filtered.csv", index=False)

print(f'Kept {len(order_products_filtered)} order-product rows '
      f'and {len(orders_sampled_filtered)} sampled orders.')

```

3. **order_product_review3:**

We first extract unique order_id from “order_products_filtered”, then create a lookup table that maps each unique order_id to a sequential review number starting at 1 (Order_id is being mapped to a sequential review_id, which is based on the order of appearance [index-like numbering] of unique order_ids), and store the mapping in a dictionary. Then we assigned each order_id a corresponding review_id based on the mapping of order_id and review_id stored in the dictionary. Lastly, we saved the new file as “order_products_with_review”

Code for order_product_review3:

```
import pandas as pd

# Load your combined order_products file
df = pd.read_csv("D:/SIIT/Project_DB_Lecture/order_products_filtered.csv")

# Step 1: create a mapping from order_id → sequential review_id
unique_orders = df["order_id"].drop_duplicates().reset_index(drop=True)
order_to_review = {order_id: idx+1 for idx, order_id in enumerate(unique_orders)}

# Step 2: assign review_id based on order_id
df["review_id"] = df["order_id"].map(order_to_review)

# Step 3: reorder columns so review_id is between product_id and add_to_cart_order
cols = list(df.columns)
# find positions
pid_idx = cols.index("product_id")
# rebuild column order
new_cols = cols[:pid_idx+1] + ["review_id"] + cols[pid_idx+1:-1] + [cols[-1]]
df = df[new_cols]

```

```
# Save result
df.to_csv("order_products_with_review.csv", index=False)

print("Added review_id column between product_id and add_to_cart_order.")
```

4. **order_order_product_mapped4:**

Here we want to solve the order_id with irregular spacing, jumping of numbers (for example. order_id: 1, 500, 2000), in “order_products_with_review” and “orders_sampled_filtered”. We then create a set that maps old order_id to a new sequential order_id, starting from 1. Afterwards, we map the old order_id in both files to the set we created and overwrite it. Lastly, we saved “order_products_with_review” in the new file as “order_product_with_review_mapped” and “orders_sampled_filtered” as “orders_sampled_filtered_mapped”.

Code for order_order_product_mapped4:

```
import pandas as pd

# Load both files
op = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review.csv")
orders = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/orders_sampled_filtered.csv")

# Collect all unique order_ids across both files
all_order_ids = pd.concat([op["order_id"],
orders["order_id"]]).drop_duplicates().reset_index(drop=True)

# Build mapping: old order_id → new continuous id
order_id_map = {old_id: new_id for new_id, old_id in enumerate(all_order_ids, start=1)}

# Overwrite the original order_id in both DataFrames
op["order_id"] = op["order_id"].map(order_id_map)
orders["order_id"] = orders["order_id"].map(order_id_map)

# Save results (overwriting old order_id with mapped values)
op.to_csv("order_products_with_review_mapped.csv", index=False)
orders.to_csv("orders_sampled_filtered_mapped.csv", index=False)

print("Replaced order_id with continuous values starting from 1 in both files.")
```

5. **products_filter5:**

Here we want to scale down the data of the products table from 50,000 instances. Foremost, we load the “product” and the “order_products_with_review_mapped” file; then compare the product_id from “product” to “order_products_with_review_mapped” and use only the one with the same product_id, which resulted in 4,500 instances. This means that these products are all products that have been ordered before. However, we want to also include and sample products that have not been ordered before, so we use the “not function” to the code we used to map product_id from “product” to “order_products_with_review_mapped” previously.

Code: for product that have not been ordered before:

```
remaining_products = products[~products["product_id"].isin(filtered_products["product_id"])]
```

This code will result in about 45,500 instances from the original “product” table. Then, we randomly sampled the data out to only 500 instances and concatenated it to the 4,500 instances we sampled before this and saved the new file as “products_filtered_plus_500”. We only sampled out 500 instances from ‘product that have not been ordered before’, because it is sufficient enough for describing the table after concatenating with the 4,500 ‘products that have been ordered before’ instances.

Code for products_filter5:

```
import pandas as pd
```

```
# Load files
```

```
products = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/data/products.csv")
```

```
order_products =
```

```
pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped.csv")
```

```
# Step 1: filter products to only those used in order_products
```

```
filtered_products = products[products["product_id"].isin(order_products["product_id"])]
```

```
# Step 2: find products not already in filtered_products set
```

```
remaining_products = products[~products["product_id"].isin(filtered_products["product_id"])]
```

```
# Step 3: randomly select 500 from remaining (adjust if fewer than 500 available)
```

```
extra_products = remaining_products.sample(n=500, random_state=42)
```

```
# Step 4: combine filtered + extra (500)
```

```
final_products = pd.concat([filtered_products, extra_products], ignore_index=True)
```

```
# Step 5: save result
final_products.to_csv("products_filtered_plus500.csv", index=False)

print(f'Created products_filtered_plus500.csv with {len(final_products)} rows "
      f'({len(filtered_products)} filtered + 500 extra unique).')
```

6. **product_order_product_map6:**

Here we want to solve the order_id with irregular spacing, jumping of numbers (for example. product_id: 1, 500, 2000), in “products_filtered_plus500” and “products_with_review_mapped”. We then create a set that maps old product_id to a new sequential product_id, starting from 1. Afterwards, we map the old product_id in both files to the set we created and overwrite it. Lastly, we saved “products_filtered_plus500” in the new file as “products_filtered_plus500_mapped” and “products_with_review_mapped” as “products_with_review_mapped2”.

Code for product_order_product_map6:

```
import pandas as pd

# Load both files
products = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/products_filtered_plus500.csv")
order_products = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped.csv")

# Collect all unique product_ids across both files
all_product_ids = pd.concat([products["product_id"], order_products["product_id"]]) \
    .drop_duplicates().reset_index(drop=True)

# Build mapping: old product_id → new continuous id
product_id_map = {old_id: new_id for new_id, old_id in enumerate(all_product_ids, start=1)}

# Overwrite product_id in both DataFrames
products["product_id"] = products["product_id"].map(product_id_map)
order_products["product_id"] = order_products["product_id"].map(product_id_map)
```

```
# Save results
products.to_csv("products_filtered_plus500_mapped.csv", index=False)
order_products.to_csv("order_products_with_review_mapped2.csv", index=False)

print("Replaced product_id with continuous values starting from 1 in both files.")
```

7. **product_with_retailer7:**

First, we loaded the file “products_filtered_plus500_mapped”, which contains the scaled-down and re-mapped product_ids. We then counted the number of rows (n) to know how many retailer_ids we need to generate.

First off, we need to understand that the customer_table has a total of 300 instances and 150 of the instances are ‘buyer-only’, which lies in the range of 1-200 instances; while the other 50 instances are both buyer and retailer. Furthermore, the instances from 201-300 are retailer only.

We then randomly selected 50 distinct numbers from the range 1–200. This number is used as a threshold combining customer (buyer only) and customer that is both the buyer and retailer.

Code:

```
prob = 0.3
mask = np.random.rand(n) < prob
```

This code created a probabilistic assignment:

- With 30% probability, a product is assigned to a retailer from the chosen set of 50 numbers.
- With 70% probability, a product is assigned a retailer_id randomly drawn from the range 201–300.

Code:

```
retailer_id = np.empty(n, dtype=int)
retailer_id[mask] = np.random.choice(chosen_numbers, size=mask.sum(), replace=True)
retailer_id[~mask] = np.random.randint(201, 301, size=(~mask).sum())
```

From this code, we generated a boolean mask of length n where each entry is True with probability (0.3). Products corresponding to True entries get their retailer from the chosen set, while the rest get their retailer from the 201–300 range.

We then added this new retailer_id column to the products_table’s DataFrame and saved the result as “products_filtered_plus500_mapped_with_retailer”.

Code for product_with_retailer7:

```

import pandas as pd
import numpy as np

# Load the file
Products
= pd.read_csv(r"D:/SIIT/Project_DB_Lecture/products_filtered_plus500_mapped.csv")

n = len(products)

# Step 1: randomly select 50 distinct numbers from 1–200
chosen_numbers = np.random.choice(range(1, 201), size=50, replace=False)

# Step 2: assign retailer_id
# For each row, flip a coin: either pick from chosen_numbers or from 201–300
# Example: 30% chance from chosen_numbers, 70% from 201–300
prob = 0.3
mask = np.random.rand(n) < prob

retailer_id = np.empty(n, dtype=int)
retailer_id[mask] = np.random.choice(chosen_numbers, size=mask.sum(), replace=True)
retailer_id[~mask] = np.random.randint(201, 301, size=(~mask).sum())

# Step 3: add column
products["retailer_id"] = retailer_id

# Save result

products.to_csv("products_filtered_plus500_mapped_with_retailer.csv", index=False)

print(f"Added retailer_id column: using 50 chosen numbers from 1–200 (reused across rows), "
      f"and 201–300 for the rest.")

```

8. **order_product_final8:**

We load the “order_products_with_review_mapped2” file. From the file, the code accidentally generates an extra column named “review_id.1”, so we just simply delete it and save the new file as “order_products_with_review_mapped_final”.

Code for order_product_final8:

```
import pandas as pd

# Load the file
df = pd.read_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped2.csv")

# Drop the unwanted column
if "review_id.1" in df.columns:
    df = df.drop(columns=["review_id.1"])

# Save back to the same file (or a new one if you prefer)
df.to_csv(r"D:/SIIT/Project_DB_Lecture/order_products_with_review_mapped_final.csv",
index=False)

print("Column 'review_id.1' has been removed.")
```

Data Dictionary

1. Aisles.csv (Master)

- **aisle_id** : This is the unique identifier for each aisle. This numerical ID is used to link products to their specific aisle.
- **aisle** : This is the name of the aisle or category within the store in a human-readable format, such as "frozen juice," "cereal," or "dry pasta."

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Aisles	aisle_id	This is the unique identifier for each aisle. Used to link products to their specific aisle.	Int unsigned	x	No		PK	
	aisle	The name of the aisle or category in a human-readable format (e.g., "frozen juice", "cereal", etc.).	varchar (50)	xxxxx Text (Lower case)	No			

2. Department.csv (Master)

- **department_id** : The unique identifier for each department. This numerical ID is used to link products and aisles to their specific department.
- **department** : The name of the major section or category where related products are grouped together, such as "bakery", "alcohol", or "breakfast".

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Department	department_id	The unique identifier for each department. This numerical ID links products (and possibly aisles) to departments.	Integer	x	No		PK	
	department	The name of the major section or product category, such as "bakery", "alcohol", or "breakfast".	Varchar(20)	xxxxxx Text (Lowercase)	No			

3. Order_products.csv (Many to Many Relationship)

- **order_id** : A unique identifier for the order.
- **product_id** : A unique identifier for the product.
- **add_to_cart_order** : A number that shows the order in which the product was added to the customer's cart.
- **reordered** : A binary flag (1 or 0) that shows if the customer has purchased this product before. 1 means they have, and 0 means this is the first time.

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Order_products	order_id	A unique identifier for the order. Links to the orders.csv file.	int unsigned	x	No		PK, FK	order_id [Orders]
	product_id	A unique identifier for the product. Links to the products.csv file.	int unsigned	x	No		PK, FK	product_id [Product]
	add_to_cart_order	Indicates the sequence in which the product was added to the customer's cart during this order.	Int unsigned	x	No			
	reordered	A binary flag: 1 if the product was ordered before by the customer, 0	bit	x	No	0, 1		
	review_id	Unique review identifier.	int unsigned	x	No		PK, FK	review_id [Review/Rating]

4. Orders.csv (Transaction)

- **order_id** : A unique ID number for each order.
- **eval_set** : To divide customer order data into three main groups for training and testing a machine learning model. Each group has the following meaning:

prior: Consists of **all past orders** from the majority of users. This data is used as a foundation to "teach" the model to learn each user's purchasing behavior.

train: Consists of the **single most recent order** from a group of users used to train the model. This data serves as the "correct answer" for the model to compare against and improve its prediction accuracy.

test: Consists of the **single most recent order** from another group of users. This data does not contain the actual products purchased (which is the data we want to predict) and is used to evaluate how accurately the model can predict.

- **order_number** : The chronological number of the order for a specific user (e.g., their 1st, 2nd, or 3rd order).
- **order_dow** : The day of the week the order was placed. Usually **0** for Sunday, **1** for Monday, etc.
- **order_hour_of_day** : The hour of the day the order was placed (in a 24-hour format).
- **day_since_prior_order** : The number of days since the user's last order. This is a missing value for a user's first order.

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Orders	order_id	A unique ID number for each order.	int unsigned	x	No		PK	
	eval_set	Indicates how the order is used in modeling: • prior: all historical orders for training.	enum ('prior', 'train', 'test')	xxxxxx	No			

		• train: latest known order used to compare predictions. • test: latest order for which products are to be predicted.						
	order_number	The sequential number of the order for the specific user (e.g., their 1st, 2nd, or 3rd order).	int unsigned	x	No			
	order_dow	The day of the week the order was placed. Usually 0 for Sunday, 1 for Monday, etc.	int	x	No	0 to 6		
	order_hour_of_day	The hour of the day the order was placed (in a 24-hour format).	int	x	No	0 to 23		
	day_since_prior_order	The number of days since the user's last order. This is a missing value for a user's first order.	float	12.3	Yes			
	customer_id	Unique customer identifier.	int unsigned	x	No		FK	customer_id [Customer]

5. Product.csv (Master)

- **product_id** : A unique number that identifies each product.
- **product_name** : The name of the product, like "Organic Turkey Burgur" or "White Pearl Onions."
- **aisle_id** : The ID number of the specific aisle where the product is located.
- **department_id** : The ID number of the department the product belongs to.

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Product	product_id	A unique number that identifies each product.	int unsigned	x	No		PK	
	product_name	The name of the product, such as "Organic Turkey Burger" or "White Pearl Onions."	varchar(150)	Xxxxx	No			
	aisle_id	The ID of the aisle where the product is located. Links to aisles.csv.	int unsigned	x	No		FK	aisle_id [Aisle]
	department_id	The ID of the department the product belongs to. Links to departments.csv.	int unsigned	x	No		FK	department_id [Department]
	retailer_ID	A unique ID number for each customer who placed the order. Links to the customer.csv file.	int unsigned	x	No		FK	customer_id [Retailer]

6. Promotion (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Promotion	code	Coupon code string.	varchar(20)	xxxxx	No		PK	
	remaining	Remaining of coupons	int unsigned	x	Yes			
	description	Details of the promotion.	varchar(255)	Xxxxxx	Yes			
	discount_type	Type (percentage, fixed_amount).	enum ('Percentage', 'Fixed_amount')	xxxxx	No			
	discount_value	Discount value.	float	x.xx	No			
	start_date	Valid-from date.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	end_date	Expiration date.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	product_id	A unique identifier for the product. Links to the products.csv file.	int unsigned	x	No		FK	product_id [Product]

7. Payment Transaction (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Payment Transaction	payment_id	Unique payment transaction ID.	int unsigned	x	No		PK	
	order_id	Associated order identifier. Links to the orders.csv file.	int unsigned	x	No		FK	order_id [Orders]
	amount	Amount paid.	float	x.xx	No			
	payment_date	When payment was made.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	customer_id	A unique ID number for each customer who placed the order. Links to the customer.csv file.	int unsigned	x	No		FK	retailer_id [retailer]

8. Invoice (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Invoice	invoice_id	Unique invoice document ID.	int unsigned	x	No		PK	
	order_id	Related order identifier. Links to the orders.csv file.	int unsigned	x	No		PK, FK	order_id [Orders]
	invoice_date	When invoice was generated.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	due_date	Payment due date.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	total_amount	Total billed amount.	float	x.xx	No			
	tax_amount	Tax portion of invoice.	float	x.xx	No			
	discount_amount	Discounts applied.	float	x.xx	Yes			
	customer_id	A unique ID number for each customer who placed the order. Links to the customer.csv file.	int unsigned	x	No		PK, FK	customer_id [Customer]
	status	Invoice status (draft, sent, paid).	Enum	Enum('Paid', 'Unpaid', 'Overdue')	No			

	created_at	When the invoice record was created.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	note	Extra note	varchar(255)	xxxxx	Yes			
	updated_at	Last update timestamp.	datetime	YYYY-MM-DDTHH:MM:SS	No			

9. Customer (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Customer	customer_id	Unique customer identifier.	Int unsigned	x	No		PK	
	first_name	Customer's given name.	varchar(50)	Xxxxx	No			
	middle_name	middle name of customer	varchar(50)	Xxxxx	Yes		No	
	last_name	Customer's family name.	varchar(50)	Xxxxx	No		No	
	email	Primary email for contact.	varchar(100)	xxxxx	No		Unique	
	phone_number	Primary phone number.	varchar(15)	xxxxx	No			
	date_of_birth	Customer's birth date.	date	YYYY-MM-DD	No			
	gender	Gender identity.	enum ('Male', 'Female', 'Other', 'Preferred not to say')	Xxxxx	No			
	registration_date	When a customer registered.	datetime	YYYY-MM-DD THH:MM:SS	No			
	last_login_date	Last login timestamp.	datetime	YYYY-MM-DD THH:MM:SS	Yes			

	nationality	nationality of customer	varchar(50)	Xxxxx	No			
--	--------------------	-------------------------	-------------	-------	----	--	--	--

10. Retailer - subtype of Customer (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Customer (Retailer)	status	Employment status (active, inactive).	enum('active','inactive')	xxxxx	No			
	account_number	The legal number for the bank account of a retailer.	vachar(20)	xxxx	No			
	account_name	The legal name for the bank account of a retailer.	vachar(100)	Xxxxx	No			
	retailer_id	Unique customer identifier.	int unsigned	x	No		PK, FK	customer_id [Customer]

11. Loyalty Membership (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership	membership_id	Unique loyalty membership ID.	int	x	No		PK	
	customer_id	Linked customer.	int	x	No		PK, FK	customer_id [Customer]
	tier_level	Membership tier (bronze, silver, gold).	enum('bronze', 'silver', 'gold')	xxxxx	No			
	points_balance	Current points balance.	Int unsigned	x	No			
	enrollment_date	When a customer enrolled.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	expiration_date	Program expiration date.	datetime	YYYY-MM-DDTHH:MM:SS	Yes			
	status	Membership status (active, expired).	enum('active', 'expired')	xxxxx	No			

12. Loyalty Membership (Bronze) - subtype of Loyalty Membership (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership (Bronze)	membership_id	Unique loyalty membership ID.	int unsigned	x	No		PK, FK	membership_id[Loyalty Membership]
	birthday_point	Number of loyalty points automatically credited to a member on their birthday anniversary each year	int unsigned	x	No		No	

13. Loyalty Membership (Silver) - subtype of Bronze (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership (Silver)	membership_id	Unique loyalty membership ID.	int unsigned	x	No		PK, FK	membership_id[Loyalty Membership (Bronze)]
	Discount_percent	Percentage discount on eligible purchases, determined by the member's tier	int unsigned	x	No		No	

14. Loyalty Membership (Gold) - subtype of Sliver (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Loyalty Membership (Gold)	membership_id	Unique loyalty membership ID.	int unsigned	x	No		PK, FK	membership_id[Loyalty Membership (Silver)]
	surprise_gift	Random member reward granted without notice	Enum (“airplane ticket”, “promo code”, “gift card”)	xxxxx	No		No	

15. Delivery (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Delivery	delivery_id	Unique delivery event ID.	int unsigned	x	No		PK	
	order_id	Order being delivered. Links to the orders.csv file.	int unsigned	x	No		FK	order_id [Orders]
	scheduled_date	When delivery is scheduled.	datetime	YYYY-MM-DDTHH:MM:SS	No			
	actual_delivery_date	When delivery actually occurred.	datetime	YYYY-MM-DDTHH:MM:SS	Yes			
	status	Delivery status (pending, in_transit, done)	enum('Pending', 'Shipped', 'Delivered', 'Cancelled')	Xxxxxx	No			

16. Inventory Record (Master)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Inventory Record	inventory_record_id	Unique inventory record ID.	int unsigned	x	No		PK	
	product_id	Product in stock.	int unsigned	x	No		FK	Product_id [Product]
	quantity_on_hand	Units physically in stock.	int unsigned	x	No			
	quantity_reserved	Units reserved (e.g., carts, orders).	int unsigned	x	No			
	quantity_available	Units available for sale.	int unsigned	x	No			
	reorder_level	Threshold to trigger restock.	int unsigned	x	No			
	reorder_quantity	Default restock quantity.	int unsigned	x	No			
	last_restock_date	When the product was last restocked.	datetime	YYYY-MM-DDTHH:MM:SS	Yes			
	inventory_status	Status (in_stock, out_of_stock, backordered)	Enum (In Stock, Out of Stock, backordered)	Xxxxx	No			

	lot_number	Supplier lot or batch number for traceability.	varchar(50)	xxxxxxxxx	Yes			
	storage_condition	Condition of where to store the product	varchar(100)	Xxxxx	Yes			

17. Review / Rating (Transaction)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Review / Rating	review_id	Unique review identifier.	int unsigned	x	No		PK	
	rating	Score from 1 to 5.	int	x	No	1-5		
	review_text	Optional text feedback.	varchar(255)	Xxxxxx	Yes			
	review_date	When a review was submitted.	datetime	YYYY-MM-DDTHH:MM:SS	No			

18. Address (Multivalued and Composite Attribute)

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Reference Table
Address	address_id	Unique identifier for each address.	int unsigned (Multivalued attribute)	x	No		PK	
	street	Street number and name.	varchar(50)	Xxxxx	No		PK	
	city	City name.	varchar(50)	Xxxxx	No		PK	
	province	Province or state.	varchar(50)	Xxxxx	No		PK	
	postal_code	Postal or ZIP code.	varchar(10)	Xxxxx	No		PK	
	country	Country name.	varchar(50)	Xxxxx	No		PK	
	house_number	House's number	varchar(20)	xxxxx	No		PK	
	customer_id	Unique customer identifier.	int unsigned	x	No		PK, FK	customer_id [Customer]

Create a database in MAMP Server (DDL)

Based on the data dictionary

--This SQL script creates a database named 'DB_Lecture_Project' if it does not already exist, and then sets it as the current database for subsequent operations.

```
CREATE DATABASE IF NOT EXISTS DB_Lecture_Project;
USE DB_Lecture_Project;
```

-- 1st table creations: “aisles”

```
CREATE TABLE IF NOT EXISTS aisles (
  aisle_ID INT unsigned not null AUTO_INCREMENT,
  aisle VARCHAR(50) NOT NULL,
  constraint PK_aisles primary key (aisle_ID)
);
```

-- 2nd table creations: “departments”

```
CREATE TABLE IF NOT EXISTS departments (
  department_ID INT unsigned not null AUTO_INCREMENT,
  department VARCHAR(20) NOT NULL,
  constraint PK_departments primary key (department_ID)
);
```

-- 3rd table creations: “customer”

```
CREATE TABLE IF NOT EXISTS customer (
  customer_ID INT unsigned not null AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  middle_name VARCHAR(50),
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL,
  phone_number VARCHAR(15) NOT NULL,
  date_of_birth DATE NOT NULL,
  gender ENUM('Male', 'Female', 'Other', 'Preferred not to say') NOT NULL,
  registration_date DATETIME NOT NULL,
  last_login_date DATETIME,
  nationality VARCHAR(50) NOT NULL, -- ???
  constraint PK_customers primary key (customer_ID),
  constraint UQ_customer_email UNIQUE (email)
);
```

-- 4th table creations: “loyalty_membership”

```
CREATE TABLE IF NOT EXISTS loyalty_membership (
  membership_ID INT unsigned not null AUTO_INCREMENT,
  customer_ID INT unsigned not null,
  tier_level ENUM('Bronze', 'Silver', 'Gold') NOT NULL,
  points_balance INT unsigned NOT NULL,
  enrollment_date DATETIME NOT NULL default CURRENT_TIMESTAMP,
  expiration_date DATETIME,
  status ENUM('Active', 'Expired') NOT NULL,
  primary key (membership_ID, customer_ID),
  CONSTRAINT FK_loyalty_membership_customer FOREIGN KEY (customer_ID)
REFERENCES customer(customer_ID)
);
```

-- 5th table creations: “bronze”

```
CREATE TABLE IF NOT EXISTS bronze (
  membership_ID INT unsigned not null,
  birthday_point INT unsigned not null,
  constraint PK_bronze primary key (membership_ID),
  CONSTRAINT FK_bronze_loyalty_membership FOREIGN KEY (membership_ID)
REFERENCES loyalty_membership(membership_ID)
);
```

-- 6th table creations: “silver”

```
CREATE table if not exists silver (
  membership_ID INT unsigned not null,
  discount_percent INT unsigned not null,
  constraint PK_silver primary key (membership_ID),
  CONSTRAINT FK_silver_bronze FOREIGN KEY (membership_ID) REFERENCES
bronze(membership_ID)
);
```

-- 7th table creations: “gold”

```
CREATE TABLE IF NOT EXISTS gold (
  membership_ID INT unsigned not null,
  surprise_gift ENUM('Airplane Ticket', 'Gift Card', 'Promotion Code') NOT NULL,
  constraint PK_gold primary key (membership_ID),
  CONSTRAINT FK_gold_silver FOREIGN KEY (membership_ID) REFERENCES
silver(membership_ID)
);
```

-- 8th table creations: “retailer”

```
CREATE TABLE IF NOT EXISTS retailer (
  retailer_ID INT unsigned not null,
  status ENUM('Active', 'Inactive') NOT NULL,
  account_number VARCHAR(20) NOT NULL,
  account_name VARCHAR(100) NOT NULL,
  constraint PK_retailer primary key (retailer_ID),
  CONSTRAINT FK_retailer_customer FOREIGN KEY (retailer_ID) REFERENCES
customer(customer_ID)
);
```

-- 9th table creations: “review”

```
CREATE TABLE IF NOT EXISTS review (
  review_ID INT unsigned not null AUTO_INCREMENT,
  rating INT NOT NULL,
  review_text VARCHAR(255),
  review_date DATETIME NOT NULL default CURRENT_TIMESTAMP,
  constraint PK_reviews primary key (review_ID),
  CONSTRAINT chk_rating_range CHECK (rating BETWEEN 1 AND 5)
);
```


-- 10th table creations: “Orders”

```

CREATE TABLE IF NOT EXISTS orders (
    order_ID INT unsigned not null AUTO_INCREMENT,
    customer_ID INT unsigned not null,
    eval_set ENUM('train','prior','test') NOT NULL,
    order_number INT unsigned NOT NULL,
    order_dow INT NOT NULL,
    order_hour_of_day INT NOT NULL,
    days_since_prior_order Float,
    constraint PK_orders primary key (order_ID),
    CONSTRAINT FK_orders_customer FOREIGN KEY (customer_ID) REFERENCES
customer(customer_ID),
    CONSTRAINT chk_order_dow_range CHECK (order_dow BETWEEN 0 AND 6),
    CONSTRAINT chk_order_hour_of_day_range CHECK (order_hour_of_day BETWEEN 0
AND 23)
);

```

-- 11th table creations: “products”

```

CREATE TABLE IF NOT EXISTS products (
    product_ID INT unsigned not null AUTO_INCREMENT,
    product_name VARCHAR(150) NOT NULL,
    aisle_ID INT unsigned NOT NULL,
    department_ID INT unsigned NOT NULL,
    retailer_ID INT unsigned NOT NULL,
    constraint PK_products primary key (product_ID),
    CONSTRAINT FK_products_aisles FOREIGN KEY (aisle_ID) REFERENCES
aisles(aisle_ID),
    CONSTRAINT FK_products_departments FOREIGN KEY (department_ID) REFERENCES
departments(department_ID),
    CONSTRAINT FK_products_retailer FOREIGN KEY (retailer_ID) REFERENCES
retailer(retailer_ID)
);

```

-- 12th table creations: “order_products”

```

CREATE TABLE IF NOT EXISTS order_products (
  order_ID INT unsigned not null,
  product_ID INT unsigned not null,
  review_ID INT unsigned not null,
  add_to_cart_order INT unsigned NOT NULL,
  reordered INT NOT NULL,
  primary key (order_ID, product_ID, review_ID),
  constraint FK_order_products_review FOREIGN KEY (review_ID) REFERENCES
review(review_ID),
  CONSTRAINT FK_order_products_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID),
  CONSTRAINT FK_order_products_products FOREIGN KEY (product_ID) REFERENCES
products(product_ID)
);

```

-- 13th table creations: “inventory_record”

```

CREATE TABLE IF NOT EXISTS inventory_record (
  inventory_ID INT unsigned not null AUTO_INCREMENT,
  product_ID INT unsigned not null,
  quantity_on_hand INT unsigned NOT NULL,
  quantity_reserved INT unsigned NOT NULL,
  quantity_available INT unsigned NOT NULL,
  reorder_level INT unsigned NOT NULL,
  reorder_quantity INT unsigned NOT NULL,
  last_restock_date DATETIME,
  inventory_status ENUM('In Stock', 'Out of Stock', 'backordered') NOT NULL,
  lot_number VARCHAR(50),
  storage_conditions VARCHAR(100),
  constraint PK_inventory_records primary key (inventory_ID),
  CONSTRAINT FK_inventory_records_products FOREIGN KEY (product_ID)
REFERENCES products(product_ID)
);

```

-- 14th table creations: “promotion”

```
CREATE TABLE IF NOT EXISTS promotion (
  code VARCHAR(20) not null,
  product_ID INT unsigned not null,
  remaining INT unsigned,
  description VARCHAR(255),
  discount_type ENUM('Percentage', 'Fixed Amount') NOT NULL,
  discount_value Float NOT NULL,
  start_date DATETIME NOT NULL,
  end_date DATETIME NOT NULL,
  constraint PK_promotion primary key (code),
  CONSTRAINT FK_promotion_products FOREIGN KEY (product_ID) REFERENCES
products(product_ID)
);
```

-- 15th table creations: “payment_transaction”

```
CREATE TABLE IF NOT EXISTS payment_transaction (
  payment_ID INT unsigned not null AUTO_INCREMENT,
  order_ID INT unsigned not null,
  customer_ID INT unsigned not null,
  payment_method ENUM('Credit Card', 'Debit Card', 'PayPal', 'Bank Transfer') NOT NULL,
  amount Float NOT NULL,
  currency VARCHAR(10) NOT NULL,
  payment_date DATETIME NOT NULL,
  status ENUM('Successful', 'Failed', 'Pending') NOT NULL,
  constraint PK_payment_transaction primary key (payment_ID),
  CONSTRAINT FK_payment_transaction_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID),
  CONSTRAINT FK_payment_transaction_retailer FOREIGN KEY (customer_ID)
REFERENCES retailer(retailer_ID)
);
```

-- 16th table creations: “invoice”

```

CREATE TABLE IF NOT EXISTS invoice (
    invoice_ID INT unsigned not null AUTO_INCREMENT,
    order_ID INT unsigned not null,
    customer_ID INT unsigned not null,
    invoice_date DATETIME NOT NULL default CURRENT_TIMESTAMP,
    due_date DATETIME NOT NULL,
    total_amount Float NOT NULL,
    tax_amount Float NOT NULL,
    discount_amount Float,
    status ENUM('Paid', 'Unpaid', 'Overdue') NOT NULL,
    created_at DATETIME NOT NULL default CURRENT_TIMESTAMP,
    updated_at DATETIME,
    note VARCHAR(255),
    primary key (invoice_ID, order_ID, customer_ID),
    CONSTRAINT FK_invoice_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID),
    CONSTRAINT FK_invoice_customer FOREIGN KEY (customer_ID) REFERENCES
customer(customer_ID)
);

```

-- 17th table creations: “delivery”

```

CREATE TABLE IF NOT EXISTS delivery (
    delivery_ID INT unsigned not null AUTO_INCREMENT,
    order_ID INT unsigned not null,
    scheduled_date DATETIME NOT NULL,
    actual_delivery_date DATETIME,
    status ENUM('Pending', 'Shipped', 'Delivered', 'Cancelled') NOT NULL,
    constraint PK_delivery primary key (delivery_ID),
    CONSTRAINT FK_delivery_orders FOREIGN KEY (order_ID) REFERENCES
orders(order_ID)
);

```

-- 18th table creations: “address”

```
CREATE TABLE IF NOT EXISTS address (  
    address_ID INT unsigned not null AUTO_INCREMENT,  
    customer_ID INT unsigned not null,  
    house_number VARCHAR(20) NOT NULL,  
    street VARCHAR(50) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    province VARCHAR(50) NOT NULL,  
    postal_code VARCHAR(10) NOT NULL,  
    country VARCHAR(50) NOT NULL,  
    primary key (address_ID, customer_ID, house_number, street, city, province, postal_code,  
country),  
    CONSTRAINT FK_address_customer FOREIGN KEY (customer_ID) REFERENCES  
customer(customer_ID)  
);
```

Example of Data Manipulation Language (DML)

1. #parser_table1

We first load the aisle data by reading the file aisles.csv from the 'files location' directory and inserting its records into the aisles table, skipping the first header row. Similarly, we will load the department data from departments.csv in the same directory and insert those records into the departments table, also skipping its header row.

Code:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/aisles.csv'

insert aisles.csv data into aisles table

```
LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_sampled_2/aisles.csv'
INTO TABLE aisles
CHARACTER SET 'utf8mb4'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/departments.csv'

insert departments.csv data into departments table

```
LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_sampled_2/departments.csv'
INTO TABLE departments
CHARACTER SET 'utf8mb4'
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

2. #parser_table2

We insert records into “customer” table, specifying which contains ‘first_name’, ‘last_name’, ‘email’, ‘phone_number’, ‘date_of_birth’, and ‘nationality’. Similarly to the customer, we will insert records into “bronze” table, providing the ‘membership_id’ and its corresponding to ‘birthday_point’ for each. For “silver” table, saving records that map a ‘membership_id’ to a ‘discount_percent’. And also “gold” table, assigning a ‘surprise_gift’ to each specified ‘membership_id’. Finally, we will insert records individual records into the ‘review’ table, storing each ‘review’s id’, ‘rating’, ‘text’, and ‘submission date’.

remark: As a matter of fact, we inserted hundred(s) of instances in our database. However, we only show the first five rows of the instances in this report to reduce the space being used.

Code:

```
INSERT INTO customer (first_name, middle_name, last_name, email, phone_number,
date_of_birth, gender, registration_date, last_login_date, nationality)
-VALUES-
('Danielle', NULL, 'Johnson', 'danielle.johnson0@example.com', '33218196001', '1979-03-23',
'Male', '2021-01-22 06:23:12', '2025-10-31 23:51:06', 'Gabonese'),
('James', 'Debra', 'Gardner', 'james.gardner1@example.com', '79402654235', '1970-09-09', 'Male',
'2021-08-28 07:25:02', '2023-03-08 23:08:18', 'Samoan'),
('Melanie', 'Cassandra', 'Roman', 'melanie.roman2@example.com', '81618495931', '1997-06-10',
'Preferred not to say', '2023-04-18 22:03:05', '2023-08-20 20:54:42', 'British'),
('Melissa', 'Michelle', 'James', 'melissa.james3@example.com', '52553419283', '1970-08-28',
'Male', '2023-02-17 12:29:24', '2025-07-01 17:04:53', 'Togolese'),
('Jeffrey', NULL, 'Nguyen', 'jeffrey.nguyen4@example.com', '30564139537', '1954-07-23',
'Preferred not to say', '2023-02-01 18:37:06', '2023-09-06 16:24:50', 'Slovenian'),
```

```
INSERT INTO bronze (membership_ID, birthday_point)
-VALUES-
(1, 765),
(2, 330),
(3, 702),
(4, 160),
(5, 172),
```

```
INSERT INTO silver (membership_ID, discount_percent)
```

```
-VALUES-
```

```
(78, 24),
```

```
(167, 6),
```

```
(105, 28),
```

```
(30, 30),
```

```
(36, 26),
```

```
INSERT INTO gold (membership_ID, surprise_gift)
```

```
-VALUES-
```

```
(35, 'Promotion Code'),
```

```
(46, 'Promotion Code'),
```

```
(157, 'Airplane Ticket'),
```

```
(158, 'Airplane Ticket'),
```

```
(139, 'Airplane Ticket'),
```

```
INSERT INTO review (review_ID, rating, review_text, review_date)
```

```
-VALUES-
```

```
(1, 1, 'Agent every development say quality throughout.', '2024-06-18 01:52:41'),
```

```
(2, 2, 'Opportunity all behavior discussion.', '2022-01-16 01:52:41'),
```

```
(3, 5, 'Ago current practice nation determine operation speak according south recently future  
choice whatever from behavior.', '2025-08-28 01:52:41'),
```

```
(4, 1, 'Grow gas enough analysis least.', '2023-01-02 01:52:41'),
```

```
(5, 5, 'Win her need stop peace technology.', '2021-10-27 01:52:41'),
```

3. #parser_table3

We first load the order data by reading the file “orders_sampled_filtered_mapped.csv” from the “file location” directory and inserting its records into the “orders” table, skipping the first header row and specifically converting any empty string for the “days_since_prior_order” column into proper NULL values. Then, we will load the product data from “products_filtered_plus500_mapped_with_retailer.csv” in the same directory and insert those records into the “products” table, also skipping its header row. Similarly, we will populate the “order_products” table by importing data from “order_products_with_review_mapped_final.csv”. Finally, we will load the inventory data by reading the inventory_record.csv file from the “file location” directory and inserting its records into the inventory_record table, skipping the header row for both of these last two imports.

Code:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/orders_sampled_filtered.csv'

insert orders_sampled_filtered.csv data into orders table

LOAD DATA LOCAL INFILE

'D:/SIIT/Project_DB_Lecture/data_sampled_2/orders_sampled_filtered_mapped.csv'

INTO TABLE orders

FIELDS TERMINATED BY ','

OPTIONALLY ENCLOSED BY ''''

LINES TERMINATED BY '\r\n'

IGNORE 1 ROWS

(order_ID, customer_ID, eval_set, order_number, order_dow, order_hour_of_day, @days)

SET days_since_prior_order = NULLIF(TRIM(@days), "");

Location of file -> 'D:/SIIT/Project_DB_Lecture/data_sampled/products_with_retailer.csv'

insert products_with_retailer.csv data into products table

LOAD DATA LOCAL INFILE

'D:/SIIT/Project_DB_Lecture/data_sampled_2/products_filtered_plus500_mapped_with_retailer.csv'

INTO TABLE products

FIELDS TERMINATED BY ','

OPTIONALLY ENCLOSED BY ''''

LINES TERMINATED BY '\n'

IGNORE 1 ROWS;

Location of file ->

'D:/SIIT/Project_DB_Lecture/data_sampled/order_products_combined_filtered.csv'

insert order_products_combined_filtered.csv data into order_products table

LOAD DATA LOCAL INFILE

'D:/SIIT/Project_DB_Lecture/data_sampled_2/order_products_with_review_mapped_final.csv'

INTO TABLE order_products

FIELDS TERMINATED BY ','

OPTIONALLY ENCLOSED BY ''''

```

LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

```

```

LOAD DATA LOCAL INFILE
'D:/SIIT/Project_DB_Lecture/data_random/inventory_record.csv'
INTO TABLE inventory_record
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

```

4. #parser_table4

We insert four new records into the “promotion” table, populating each with its unique promotion code, associated product ID, remaining quantity, description, discount details (type and value), and its active date range (start and end dates).

remark: As a matter of fact, we inserted hundred(s) of instances in our database. However, we only show the first five rows of the instances in this report to reduce the space being used.

Code:

```

INSERT INTO promotion (code, product_ID, remaining, description, discount_type,
discount_value, start_date, end_date)
VALUES
('PROMO0001', 913, 381, 'Agent every development say quality throughout.', 'Fixed Amount',
45.4, '2025-07-22 02:10:18', '2025-11-21 02:10:18'),
('PROMO0002', 205, 112, 'Opportunity all behavior discussion.', 'Fixed Amount', 9.09,
'2025-10-20 02:10:18', '2025-12-17 02:10:18'),
('PROMO0003', 2254, 78, 'Ago current practice nation determine operation speak according
south.', 'Fixed Amount', 31.84, '2025-07-26 02:10:18', '2025-11-30 02:10:18'),
('PROMO0004', 2007, 305, 'Future choice whatever from behavior benefit suggest page southern
role movie win her.', 'Fixed Amount', 48.7, '2025-10-30 02:10:18', '2025-12-28 02:10:18'),

```

5. #parser_table5

We first load the payment transaction data by reading the “payment_transaction.csv” file from the “file location” directory and inserting its records into the “payment_transaction” table, skipping the header row and trimming any extra whitespace from the “status” field. Then, we will load the invoice table; this process also skips the grader and converts any empty strings for the “note” and “updated_at” columns into proper NULL values. Finally, we will import the delivery data from “delivery.csv”, loading it into the “delivery” table while skipping its header, converting empty strings for “actual_delivery_date” into NULL, and trimming whitespace from the “status” field.

Code:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

```
LOAD DATA LOCAL INFILE
'D:/SIIT/Project_DB_Lecture/data_random/payment_transaction.csv'
INTO TABLE payment_transaction
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(payment_ID, order_ID, customer_ID, amount, payment_date, @status)
SET status = TRIM(@status);
```

```
LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_random/invoice.csv'
INTO TABLE invoice
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(invoice_ID,order_ID,customer_ID,invoice_date,due_date,total_amount,tax_amount,discount_a
mount,status,created_at,@updated_at,note)
SET
note = NULLIF(note, ""),
updated_at = NULLIF(@updated_at, "");
```

```

LOAD DATA LOCAL INFILE 'D:/SIIT/Project_DB_Lecture/data_random/delivery.csv'
INTO TABLE delivery
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(delivery_ID,order_ID,scheduled_date,@actual_delivery_date,@status)
SET
  actual_delivery_date = NULLIF(@actual_delivery_date, ""),
  status = TRIM(@status);

```

6. #parser_table6

We first insert data into the “address” table, adding five new address records, each complete with its associated “customer_id”, full street details contain of ‘house_number’, ‘street’, ‘city’, ‘province’, ‘postal_code’, ‘country’, and creation timestamp.

remark: As a matter of fact, we inserted hundred(s) of instances in our database. However, we only show the first five rows of the instances in this report to reduce the space being used.

Code:

```

INSERT INTO address (address_ID, customer_ID, house_number, street, city, province,
postal_code, country) VALUES
(1, 1, '545', 'Sangchuto Rd', 'Tha Muang', 'Kanchanaburi', '71110', 'Thailand'),
(2, 1, '645/2', 'Khleng Luang Rd', 'Khleng Luang', 'Pathum Thani', '12120', 'Thailand'),
(3, 1, '267/1', 'Chokchai-Dej Udom Rd', 'Nang Rong', 'Buri Ram', '31110', 'Thailand'),
(4, 2, '485/A', 'Si Ayutthaya Rd', 'Mueang Ayutthaya', 'Ayutthaya', '13000', 'Thailand'),
(5, 3, '619/1', 'Jerm Jom Phon Rd', 'Si Racha', 'Chonburi', '20110', 'Thailand'),

```

Random data record using Python

1. Customer

Customer_random code Explanation:

In this code, generates 300 fake customer records using the **Faker** library and Python's **random** module. The faker instance is seeded with 42 to make the results consistent every time the code runs.

We use the Faker library to generate data because the Faker library is used for generating text-type data, which aligns with the customer data we need.

Customer_random code:

```
from faker import Faker
import random
```

```
fake = Faker()
Faker.seed(42)
random.seed(42)
```

```
genders = ['Male', 'Female', 'Other', 'Preferred not to say']
```

```
nationalities = [
    "Afghan", "Albanian", "Algerian", "American", "Andorran", "Angolan",
    "Argentine", "Armenian", "Australian", "Austrian", "Azerbaijani",
    "Bahamian", "Bahraini", "Bangladeshi", "Barbadian", "Belarusian",
    "Belgian", "Belizean", "Beninese", "Bhutanese", "Bolivian",
    "Bosnian", "Brazilian", "British", "Bruneian", "Bulgarian",
    "Burkinabé", "Burmese", "Burundian", "Cambodian", "Cameroonian",
    "Canadian", "Cape Verdean", "Central African", "Chadian", "Chilean",
    "Chinese", "Colombian", "Comorian", "Congolese", "Costa Rican",
    "Croatian", "Cuban", "Cypriot", "Czech", "Danish", "Djiboutian",
    "Dominican", "Dutch", "East Timorese", "Ecuadorian", "Egyptian",
    "Emirati", "English", "Equatorial Guinean", "Eritrean", "Estonian",
    "Ethiopian", "Fijian", "Filipino", "Finnish", "French", "Gabonese",
    "Gambian", "Georgian", "German", "Ghanaian", "Greek", "Grenadian",
    "Guatemalan", "Guinean", "Guyanese", "Haitian", "Honduran",
    "Hungarian", "Icelandic", "Indian", "Indonesian", "Iranian",
    "Iraqi", "Irish", "Israeli", "Italian", "Ivorian", "Jamaican",
    "Japanese", "Jordanian", "Kazakhstani", "Kenyan", "Korean",
    "Kuwaiti", "Kyrgyzstani", "Laotian", "Latvian", "Lebanese",
```

"Liberian", "Libyan", "Lithuanian", "Luxembourgish", "Macedonian",
 "Malagasy", "Malawian", "Malaysian", "Maldivian", "Malian",
 "Maltese", "Marshallese", "Mauritanian", "Mauritian", "Mexican",
 "Micronesia", "Moldovan", "Monegasque", "Mongolian", "Montenegrin",
 "Moroccan", "Mozambican", "Namibian", "Nepalese", "New Zealander",
 "Nicaraguan", "Nigerian", "Nigerien", "Norwegian", "Omani",
 "Pakistani", "Palauan", "Palestinian", "Panamanian",
 "Papua New Guinean", "Paraguayan", "Peruvian", "Polish",
 "Portuguese", "Qatari", "Romanian", "Russian", "Rwandan",
 "Salvadoran", "Samoan", "San Marinese", "São Toméan",
 "Saudi Arabian", "Scottish", "Senegalese", "Serbian", "Seychellois",
 "Sierra Leonean", "Singaporean", "Slovak", "Slovenian",
 "Solomon Islander", "Somali", "South African", "Spanish",
 "Sri Lankan", "Sudanese", "Surinamese", "Swazi", "Swedish",
 "Swiss", "Syrian", "Taiwanese", "Tajikistani", "Tanzanian", "Thai",
 "Togolese", "Tongan", "Trinidadian", "Tunisian", "Turkish",
 "Turkmen", "Tuvaluan", "Ugandan", "Ukrainian", "Uruguayan",
 "Uzbekistani", "Venezuelan", "Vietnamese", "Welsh", "Yemeni",
 "Zambian", "Zimbabwean"

]

```
rows = []
for i in range(300):
    first_name = fake.first_name()
    middle_name = fake.first_name() if random.random() < 0.3 else None
    last_name = fake.last_name()
    email = f'{first_name.lower()}.{last_name.lower()}@example.com'
    phone_number = fake.msisdn()[1:]
    date_of_birth = fake.date_of_birth(minimum_age=18, maximum_age=80)
    gender = random.choice(genders)
    registration_date = fake.date_time_between(start_date="-5y", end_date="now")
    last_login_date = (
        fake.date_time_between(start_date=registration_date, end_date="now").strftime("%Y-%m-%d
%H:%M:%S")
        if random.random() < 0.8 else None
    )
    nationality = random.choice(nationalities)

    row = f'({first_name}, " \
    f'NULL' if middle_name is None else repr(middle_name)}, " \
```

```

f"{last_name}', '{email}', '{phone_number}', '{date_of_birth}', " \
f"{gender}', '{registration_date.strftime('%Y-%m-%d %H:%M:%S')}', " \
f"{'NULL' if last_login_date is None else repr(last_login_date)}, " \
f"{'nationality}')"
rows.append(row)

sql = "INSERT INTO customer (first_name, middle_name, last_name, email, phone_number,
date_of_birth, gender, registration_date, last_login_date, nationality)\nVALUES\n"
sql += ",\n".join(rows) + ";\n"

print(sql)

```

2. Loyalty_membership

Membership_random code Explanation:

This code generates loyalty membership data. It creates 200 membership records with different tier levels (Bronze, Silver, Gold) and realistic fields such as points balance, enrollment dates, and status. All members are inserted into the Bronze table, then 100 of them are randomly selected as Silver members, and 50 of those are further selected as Gold members. Finally, the script prints SQL INSERT statements for the four related tables: loyalty_membership, bronze, silver, and gold.

The reason we generated 200 loyalty membership records is because we have a total of 300 customers. Therefore, having 200 loyalty memberships is a number that makes sense and is realistically reasonable. 200 instances is sufficient for the loyalty_membership table, as having more instances would not impact the results.

Membership_random code:

```
import random
from datetime import datetime, timedelta

random.seed(42)

# Step 1: Generate 200 loyalty_membership entries
tier_levels = ['Bronze', 'Silver', 'Gold']
statuses = ['Active', 'Expired']
customer_ids = random.sample(range(1, 301), 200)

loyalty_rows = []
bronze_rows = []
silver_rows = []
gold_rows = []

for i, customer_id in enumerate(customer_ids):
    membership_id = i + 1
    tier = random.choice(tier_levels)
    points = random.randint(0, 10000)
    enrollment_date = datetime.now() - timedelta(days=random.randint(0, 1825))
    expiration_date = (
        enrollment_date + timedelta(days=random.randint(365, 1825))
        if random.random() < 0.7 else None
    )
```



```

status = 'Expired' if expiration_date and expiration_date < datetime.now() else 'Active'

loyalty_rows.append(
    f'({membership_id}, {customer_id}, {tier}, {points},
    '{expiration_date.strftime("%Y-%m-%d %H:%M:%S")}', "
    f'NULL' if expiration_date is None else repr(expiration_date.strftime("%Y-%m-%d
    %H:%M:%S"))}, {status})'
)
# Step 2: Bronze table (all loyalty_membership entries)
birthday_point = random.randint(100, 1000)
bronze_rows.append(f'({membership_id}, {birthday_point})')

# Step 3: Silver table (100 random bronze entries)
silver_ids = random.sample([row.split(",")[0][1:] for row in bronze_rows], 100)
for sid in silver_ids:
    discount = random.randint(5, 30)
    silver_rows.append(f'({sid}, {discount})')

# Step 4: Gold table (50 random silver entries)
gold_ids = random.sample([row.split(",")[0][1:] for row in silver_rows], 50)
gifts = ['Airplane Ticket', 'Gift Card', 'Promotion Code']
for gid in gold_ids:
    gift = random.choice(gifts)
    gold_rows.append(f'({gid}, {gift})')

# Final SQL
print("-- loyalty_membership")
print("INSERT INTO loyalty_membership (membership_ID, customer_ID, tier_level,
points_balance, enrollment_date, expiration_date, status)\nVALUES")
print(",\n".join(loyalty_rows) + ";")

print("\n-- bronze")
print("INSERT INTO bronze (membership_ID, birthday_point)\nVALUES")
print(",\n".join(bronze_rows) + ";")

print("\n-- silver")
print("INSERT INTO silver (membership_ID, discount_percent)\nVALUES")
print(",\n".join(silver_rows) + ";")

print("\n-- gold")

```

```
print("INSERT INTO gold (membership_ID, surprise_gift)\nVALUES")  
print("\n".join(gold_rows) + ";")
```

3. Retailor

Retailor_random code Explanation:

This code loads a product dataset and extracts unique retailer IDs. From those IDs, it randomly selects 150 retailers and generates fake retailer information using the Faker library, including status (Active/Inactive), account number, and account name. Finally, it formats the generated data into an SQL INSERT statement for the retailer table and prints it out.

Since our dataset contains a total of 300 customers, and customers can include both buyers and retailers — with some individuals acting as both — we decided to generate retailer data for 150 of them to better reflect a realistic scenario and it is sufficient for the retailer_random table, as having more instances would not impact the results.

Retailor_random code:

```
import pandas as pd

# Load the CSV
df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/products_filtered_plus500_mapped
_with_retailer.csv')

# Extract unique retailer_IDs
retailer_ids = df['retailer_id'].dropna().unique()
retailer_ids = [int(rid) for rid in retailer_ids]

import random
random.seed(42)

sampled_ids = random.sample(retailer_ids, 150)

from faker import Faker
fake = Faker()

rows = []
for rid in sampled_ids:
    status = random.choice(['Active', 'Inactive'])
    account_number = str(random.randint(1000000000, 9999999999))
    account_name = fake.company()
    rows.append(f'({rid}, '{status}', '{account_number}', '{account_name}'))

sql = "INSERT INTO retailer (retailer_id, status, account_number, account_name)\nVALUES\n"
```

```
sql += ",\n".join(rows) + ";
```

```
print(sql)
```

4. Review

Review_random code Explanation:

- This code generates 951 reviews with ratings (1–5), optional review text, and dates. Using `random.seed(42)` and `Faker.seed(42)` ensures reproducibility, and the output is formatted as an SQL INSERT statement ready for the review table.
- Review text has an 80% chance of being generated
“`review_text = fake.sentence(...)` if `random.random() < 0.8` else `'NULL'`”
Simulates real-world cases where a review may have a rating but no text.
- Review date is randomized within the last 5 years
“`datetime.now() - timedelta(days=random.randint(0, 1825))`”
Adds time variability for trends and time-based analysis.

Review_random code:

```
import random
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
random.seed(42)
Faker.seed(42)

reviews = []
for i in range(1, 951):
    rating = random.randint(1, 5)
    review_text = fake.sentence(nb_words=random.randint(5, 15)) if random.random() < 0.8 else 'NULL'
    review_date = datetime.now() - timedelta(days=random.randint(0, 1825))

    row = f'({i}, {rating}, " \
        f'{\'NULL\' if review_text == \'NULL\' else repr(review_text)}', " \
        f'{review_date.strftime(\'%Y-%m-%d %H:%M:%S\')})"'
    reviews.append(row)

sql = "INSERT INTO review (review_ID, rating, review_text, review_date)\nVALUES\n"
sql += ",\n".join(reviews) + ";"

print(sql)
```

5. Inventory_record

inventory_record_review Code Explanation:

This code is designed to generate synthetic data records for the **inventory_record** table by randomly assigning values to each attribute. The **inventory_record** table has a **one-to-one relationship** with the **product** table, which originates from a Kaggle dataset. As such, the **product_id** attribute in the **inventory_record** table serves as a **foreign key (FK)** referencing the **product_id** in the **product** table. To maintain referential integrity between the two tables, the number of records generated in the **inventory_record** table must match the number of **product_id** entries in the **product** table, which totals **5084**. Each generated record must use a **product_id** that corresponds exactly to one found in the **product** table.

Code: for i in range(1, 5085):

This requirement is fulfilled by the line of code `for i in range(1, 5085):`, which iterates 5084 times to create 5084 records. Each iteration assigns the loop index `i` as the **product_id**, ensuring that the data in both tables remains consistent and properly linked.

Inventory_record_review Code:

```
import random
import csv
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
random.seed(42)
Faker.seed(42)

# Open CSV file for writing
with open('inventory_record.csv', mode='w', newline="", encoding='utf-8') as file:
    writer = csv.writer(file)

    # Write header
    writer.writerow([
        'inventory_ID', 'product_ID', 'quantity_on_hand', 'quantity_reserved',
        'quantity_available', 'reorder_level', 'reorder_quantity',
        'last_restock_date', 'inventory_status', 'lot_number', 'storage_conditions'
```

)

```
# Write 5084 rows
for i in range(1, 5085):
    quantity_on_hand = random.randint(0, 500)
    quantity_reserved = random.randint(0, quantity_on_hand)
    quantity_available = quantity_on_hand - quantity_reserved
    reorder_level = random.randint(10, 100)
    reorder_quantity = random.randint(50, 200)
    last_restock_date = (datetime.now() - timedelta(days=random.randint(0,
365))).strftime('%Y-%m-%d %H:%M:%S')
    inventory_status = (
        'Out of Stock' if quantity_on_hand == 0 else
        'backordered' if quantity_available < reorder_level else
        'In Stock'
    )
    lot_number = fake.bothify(text='LOT-#####')
    storage_conditions = random.choice([
        'Cool and dry place', 'Refrigerated', 'Room temperature',
        'Keep away from sunlight', 'Humidity controlled'
    ])

writer.writerow([
    i, i, quantity_on_hand, quantity_reserved, quantity_available,
    reorder_level, reorder_quantity, last_restock_date,
    inventory_status, lot_number, storage_conditions
])
```

6. Promotion

promotion_random Code Explanation:

This code generates synthetic data records for the **promotion** table by randomly assigning values to each attribute. The **promotion** table has a **one-to-many (1:M)** relationship (product 1 : M promotion) with the **product** table, which is based on a dataset from Kaggle. As a result, the promotion table includes a **product_id** attribute that serves as a **foreign key (FK)** referencing the product table. In real-world business scenarios, promotions are typically applied to a limited number of products rather than the entire catalog. Given that the Kaggle dataset contains **5084 product_id**, the code randomly selects **100 product_id** from the range 1 to 5084 to simulate this selective promotion strategy.

The line of code **product_ids = random.sample(range(1, 5085), 100)** performs this sampling, ensuring 100 unique **product_id** are chosen.

Then, the line **for i, pid in enumerate(product_ids):** iterates through each selected **product_id**, using **i** as the promotion index and **pid** as the associated product. This loop is used to generate 100 promotion records, each linked to a different product.

promotion_random Code:

```
import random
from faker import Faker
from datetime import datetime, timedelta

fake = Faker()
random.seed(42)
Faker.seed(42)

# Step 1: Sample 100 distinct product_IDs
product_ids = random.sample(range(1, 5085), 100)

# Step 2: Generate promotion rows
rows = []
for i, pid in enumerate(product_ids):
    code = f"PROMO{i+1:04d}"
    remaining = random.randint(0, 500)
    description = fake.sentence(nb_words=random.randint(5, 12))
    discount_type = random.choice(['Percentage', 'Fixed Amount'])
```



```

    discount_value = round(random.uniform(5, 50), 2) if discount_type == 'Fixed Amount' else
random.randint(5, 50)
    start_date = datetime.now() - timedelta(days=random.randint(0, 180))
    end_date = start_date + timedelta(days=random.randint(30, 180))

    row = f"('{code}', {pid}, {remaining}, '{description}', '{discount_type}', {discount_value}, " \
        f"'{start_date.strftime('%Y-%m-%d %H:%M:%S')}', '{end_date.strftime('%Y-%m-%d %H:%M:%S')}')"
    rows.append(row)

# Step 3: Build SQL
sql = "INSERT INTO promotion (code, product_ID, remaining, description, discount_type,
discount_value, start_date, end_date)\nVALUES\n"
sql += ",\n".join(rows) + ";"

print(sql)

```

7. Invoice

invoice_random Code Explanation:

This process involves generating data records for the **invoice** table by randomly assigning values to each attribute. The **invoice** table is considered a **weak entity**, meaning it cannot exist independently and must be associated with a parent entity—in this case, the **orders** table, which originates from a Kaggle dataset. The relationship between **invoice** and **orders** is **one-to-one (1:1)**, meaning each invoice corresponds to exactly one order, and each order has one invoice. Therefore, the **invoice** table includes the attribute **order_id** as a **foreign key (FK)** referencing the **orders** table.

Additionally, the **invoice** table has a **one-to-many (1:M)** relationship with the **customer** table, where each customer can have multiple invoices, but each invoice is linked to only one customer. As a result, the **invoice** table also includes the attribute **customer_id**, which serves as a foreign key referencing the **customer** table.

To maintain referential integrity, the number of invoice records must be equal to the number of order records, and each **order_id** used in the invoice table must match an existing **order_id** in the orders table. Likewise, the **customer_id** values in the invoice table must be consistent with those found in the orders table, which themselves originate from the **customer 1:M orders** relationship—where **customer_id** in the orders table is a foreign key referencing the customer table.

Invoice random Code:

```
import pandas as pd

import random

from faker import Faker

from datetime import datetime, timedelta

# Load the orders CSV

orders_df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/orders_sampled_filtered_mapped.csv')
```

```

# Normalize column names

orders_df.columns = orders_df.columns.str.strip().str.lower().str.replace(' ', '_')


# Filter valid rows with both order_id and customer_id

orders_df = orders_df.dropna(subset=['order_id', 'customer_id'])

orders_df['order_id'] = orders_df['order_id'].astype(int)

orders_df['customer_id'] = orders_df['customer_id'].astype(int)


# Initialize Faker

fake = Faker()

random.seed(42)

Faker.seed(42)


# Generate invoice data

records = []

for i, row in orders_df.iterrows():

    invoice_id = i + 1

    order_id = row['order_id']

    customer_id = row['customer_id']

    invoice_date = datetime.now() - timedelta(days=random.randint(0, 60))

    due_date = invoice_date + timedelta(days=random.randint(15, 45))

    total_amount = round(random.uniform(500, 10000), 2)

    tax_amount = round(total_amount * 0.07, 2)

```

```

discount_amount = round(random.uniform(0, total_amount * 0.1), 2) if random.random() <
0.5 else 0.0

status = random.choice(['Paid', 'Unpaid', 'Overdue'])

created_at = invoice_date

updated_at = invoice_date + timedelta(days=random.randint(0, 30)) if random.random() < 0.7
else ""

note = fake.sentence(nb_words=random.randint(5, 12)) if random.random() < 0.5 else ""

records.append([

    invoice_id, order_id, customer_id,

    invoice_date.strftime('%Y-%m-%d %H:%M:%S'),

    due_date.strftime('%Y-%m-%d %H:%M:%S'),

    total_amount, tax_amount, discount_amount,

    status, created_at.strftime('%Y-%m-%d %H:%M:%S'),

    updated_at if updated_at == "" else updated_at.strftime('%Y-%m-%d %H:%M:%S'),

    note

])

# Save to CSV

df_out = pd.DataFrame(records, columns=[

    'invoice_ID', 'order_ID', 'customer_ID', 'invoice_date', 'due_date',

    'total_amount', 'tax_amount', 'discount_amount', 'status',

    'created_at', 'updated_at', 'note'

])

df_out.to_csv('invoice.csv', index=False, encoding='utf-8')

```


8. Payment transaction

payment_transaction_random Code Explanation:

This program generates synthetic data records for the **payment_transaction** table by randomly assigning values to each attribute, while referencing real data from three existing tables:

order_products, **products**, and **invoice**. The purpose is to simulate payments made to retailers based on customer orders.

The **payment_transaction** table has a **one-to-many (1:M)** relationship with the **orders** table, meaning each order can be associated with multiple payment transactions. It also has a **one-to-many (1:M)** relationship with the **retailer** table, since each retailer can receive multiple payments. Therefore, the **payment_transaction** table includes the attribute **order_id** as a foreign key referencing the **orders** table, and the attribute **retailer_id** (labeled as **customer_ID** in the output) referencing the **retailer** table.

From the **invoice** table, the attribute **total_amount** is used to determine the total value of an order. In the **payment_transaction** table, this value is distributed into the attribute **amount**, which represents the portion of the payment allocated to each retailer linked to that order. The distribution is randomized but ensures that the sum of all amounts equals 70% of the invoice's total amount (to simulate platform fees or deductions).

payment_transaction_random Code:

```
import pandas as pd

import random

from faker import Faker

from datetime import datetime, timedelta

# Load CSVs

order_products_df =
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/order_products_with_review_mapped_final.csv')
```

```
products_df =  
pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_sampled_2/products_filtered_plus500_mapped  
_with_retailer.csv')  
  
invoice_df = pd.read_csv(r'D:/SIIT/Project_DB_Lecture/data_random/invoice.csv')  
  
# Normalize column names  
  
order_products_df.columns = order_products_df.columns.str.strip().str.lower().str.replace(' ', '_')  
products_df.columns = products_df.columns.str.strip().str.lower().str.replace(' ', '_')  
invoice_df.columns = invoice_df.columns.str.strip().str.lower().str.replace(' ', '_')  
  
# Merge order_products with products to get retailer_id  
  
merged_df = order_products_df.merge(products_df[['product_id', 'retailer_id']], on='product_id',  
how='inner')  
  
merged_df = merged_df.dropna(subset=['retailer_id'])  
  
merged_df['retailer_id'] = merged_df['retailer_id'].astype(int)  
  
# Merge with invoice to get total_amount  
  
merged_df = merged_df.merge(invoice_df[['order_id', 'total_amount']], on='order_id', how='left')  
  
merged_df = merged_df.dropna(subset=['total_amount'])  
  
# Generate payment_transaction data  
  
fake = Faker()  
  
random.seed(42)  
  
Faker.seed(42)
```

```

records = []

payment_id = 1

# Group by order_id
for order_id, group in merged_df.groupby('order_id'):

    total_amount = float(group['total_amount'].iloc[0])

    target_amount = round(total_amount * 0.7, 2)

    num_products = len(group)

    weights = [random.random() for _ in range(num_products)]

    total_weight = sum(weights)

    splits = [(w / total_weight) * target_amount for w in weights]

    splits[-1] = target_amount - sum(splits[:-1])

    splits = [round(x, 2) for x in splits]

    for i, (_, row) in enumerate(group.iterrows()):

        retailer_id = int(row['retailer_id'])

        payment_date = datetime.now() - timedelta(days=random.randint(0, 365))

        status = random.choice(['Successful', 'Failed', 'Pending'])

        records.append([

            payment_id, order_id, retailer_id, splits[i],

            payment_date.strftime('%Y-%m-%d %H:%M:%S'), status

```



```
)  
  
    payment_id += 1  
  
# Save to CSV  
  
df_out = pd.DataFrame(records, columns=[  
    'payment_ID', 'order_ID', 'customer_ID', 'amount',  
    'payment_date', 'status'  
)  
  
df_out.to_csv(r'D:/SIIT/Project_DB_Lecture/data_random/payment_transaction.csv',  
index=False, encoding='utf-8')
```

9. Delivery

Delivery_random code Explanation:

This code will result in an error because there is a specific local file path. To solve this, please download the file and change the file path to where you downloaded the file on your local device.

```
“invoice_df = pd.read_csv(r'D:/SIIT/Project_DB_Lecture/invoice.csv’)”
```

This script generates delivery records based on existing invoice data. For each invoice, it creates a delivery with a scheduled date (1–10 days after the invoice)

```
“scheduled_date = row['invoice_date'] + timedelta(days=random.randint(1, 10))”
```

, a status (Pending, Shipped, Delivered, or Cancelled)

```
“status = random.choice(['Pending', 'Shipped', 'Delivered', 'Cancelled'])”
```

, and an actual delivery date calculated according to the status.

```
“actual_delivery_date = "
  if status == 'Delivered':
    actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 5))
  elif status == 'Shipped':
    actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 3))
  elif status == 'Cancelled':
    actual_delivery_date = scheduled_date”
```

The resulting dataset is saved as delivery.csv for use in the database.

Delivery_random code:

```
import pandas as pd
import random
from datetime import datetime, timedelta

# Load invoice data
invoice_df = pd.read_csv(r'D:/SIIT/Project_DB_Lecture/invoice.csv')
```

```

# Normalize column names
invoice_df.columns = invoice_df.columns.str.strip().str.lower().str.replace(' ', '_')

# Filter valid order_IDs and invoice_dates
invoice_df = invoice_df.dropna(subset=['order_id', 'invoice_date'])
invoice_df['order_id'] = invoice_df['order_id'].astype(int)
invoice_df['invoice_date'] = pd.to_datetime(invoice_df['invoice_date'])

# Generate delivery records
random.seed(42)
records = []
for i, row in invoice_df.iterrows():
    delivery_id = i + 1
    order_id = row['order_id']
    scheduled_date = row['invoice_date'] + timedelta(days=random.randint(1, 10))
    status = random.choice(['Pending', 'Shipped', 'Delivered', 'Cancelled'])

    actual_delivery_date = ""
    if status == 'Delivered':
        actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 5))
    elif status == 'Shipped':
        actual_delivery_date = scheduled_date + timedelta(days=random.randint(1, 3))
    elif status == 'Cancelled':
        actual_delivery_date = scheduled_date

    records.append([
        delivery_id,
        order_id,
        scheduled_date.strftime('%Y-%m-%d %H:%M:%S'),
        actual_delivery_date.strftime('%Y-%m-%d %H:%M:%S') if actual_delivery_date else "",
        status
    ])

# Save to CSV
df_out = pd.DataFrame(records, columns=[
    'delivery_ID', 'order_ID', 'scheduled_date', 'actual_delivery_date', 'status'
])
df_out.to_csv('delivery.csv', index=False, encoding='utf-8')

```

10.Address

Address_random code Explanation:

This code generates realistic Thai addresses for 300 customers. Each customer can have 1–3 addresses randomly selected from a pool of 40 Thai districts (25 Mueang and 15 non-Mueang). House numbers are randomly generated. The output is formatted as an SQL INSERT statement for the address table, ready for database insertion.

Address_random code:

```
import random
from datetime import datetime

# Thai address pool (25 Mueang + 15 non-Mueang)
# ... [Insert the full 40-address list from earlier here] ...

thai_addresses = [
    #  25 Mueang Districts
    {"city": "Mueang Chiang Mai", "province": "Chiang Mai", "street": "Nimmanhaemin Rd",
     "postal_code": "50200", "country": "Thailand"},
    {"city": "Mueang Nakhon Ratchasima", "province": "Nakhon Ratchasima", "street":
     "Mittraphap Rd", "postal_code": "30000", "country": "Thailand"},
    {"city": "Mueang Khon Kaen", "province": "Khon Kaen", "street": "Sri Chan Rd",
     "postal_code": "40000", "country": "Thailand"},
    {"city": "Mueang Phuket", "province": "Phuket", "street": "Yaowarat Rd", "postal_code":
     "83000", "country": "Thailand"},
    {"city": "Mueang Chonburi", "province": "Chonburi", "street": "Sukhumvit Rd",
     "postal_code": "20000", "country": "Thailand"},
    {"city": "Mueang Udon Thani", "province": "Udon Thani", "street": "Prajak Rd",
     "postal_code": "41000", "country": "Thailand"},
    {"city": "Mueang Chiang Rai", "province": "Chiang Rai", "street": "Banpapraikan Rd",
     "postal_code": "57000", "country": "Thailand"},
    {"city": "Mueang Nakhon Pathom", "province": "Nakhon Pathom", "street": "Phetkasem Rd",
     "postal_code": "73000", "country": "Thailand"},
    {"city": "Mueang Surat Thani", "province": "Surat Thani", "street": "Talad Mai Rd",
     "postal_code": "84000", "country": "Thailand"},
    {"city": "Mueang Ayutthaya", "province": "Ayutthaya", "street": "Si Ayutthaya Rd",
     "postal_code": "13000", "country": "Thailand"},

```

```

{"city": "Mueang Sukhothai", "province": "Sukhothai", "street": "Sukhothai Rd",
"postal_code": "64000", "country": "Thailand"},
{"city": "Mueang Lampang", "province": "Lampang", "street": "Thipchang Rd",
"postal_code": "52000", "country": "Thailand"},
{"city": "Mueang Phitsanulok", "province": "Phitsanulok", "street": "Boromtrailokkanat Rd",
"postal_code": "65000", "country": "Thailand"},
{"city": "Mueang Nakhon Si Thammarat", "province": "Nakhon Si Thammarat", "street":
"Ratchadamnoen Rd", "postal_code": "80000", "country": "Thailand"},
{"city": "Mueang Trang", "province": "Trang", "street": "Ruangrit Rd", "postal_code":
"92000", "country": "Thailand"},
{"city": "Mueang Narathiwat", "province": "Narathiwat", "street": "Narathiwat Rd",
"postal_code": "96000", "country": "Thailand"},
{"city": "Mueang Pattani", "province": "Pattani", "street": "Yarang Rd", "postal_code":
"94000", "country": "Thailand"},
{"city": "Mueang Yala", "province": "Yala", "street": "Siri Rat Rd", "postal_code": "95000",
"country": "Thailand"},
{"city": "Mueang Nakhon Sawan", "province": "Nakhon Sawan", "street": "Sawanvitee Rd",
"postal_code": "60000", "country": "Thailand"},
{"city": "Mueang Ratchaburi", "province": "Ratchaburi", "street": "Phetkasem Rd",
"postal_code": "70000", "country": "Thailand"},
{"city": "Mueang Chanthaburi", "province": "Chanthaburi", "street": "Sukhumvit Rd",
"postal_code": "22000", "country": "Thailand"},
{"city": "Mueang Tak", "province": "Tak", "street": "Intharakiri Rd", "postal_code": "63000",
"country": "Thailand"},
{"city": "Mueang Kalasin", "province": "Kalasin", "street": "Pho Klang Rd", "postal_code":
"46000", "country": "Thailand"},
{"city": "Mueang Kamphaeng Phet", "province": "Kamphaeng Phet", "street":
"Ratchadamnoen Rd", "postal_code": "62000", "country": "Thailand"},
{"city": "Mueang Ubon Ratchathani", "province": "Ubon Ratchathani", "street": "Sapphasit
Rd", "postal_code": "34000", "country": "Thailand"},

```

15 Non-Mueang Districts

```

{"city": "Si Racha", "province": "Chonburi", "street": "Jerm Jom Phon Rd", "postal_code":
"20110", "country": "Thailand"},
{"city": "Bang Pa-in", "province": "Ayutthaya", "street": "U-Thong Rd", "postal_code":
"13160", "country": "Thailand"},
{"city": "Mae Sai", "province": "Chiang Rai", "street": "Phahonyothin Rd", "postal_code":
"57130", "country": "Thailand"},
{"city": "Hang Dong", "province": "Chiang Mai", "street": "Sanpatong Rd", "postal_code":
"50230", "country": "Thailand"},

```

```

{"city": "San Sai", "province": "Chiang Mai", "street": "Maejo Rd", "postal_code": "50210",
"country": "Thailand"},
{"city": "Ban Bueng", "province": "Chonburi", "street": "Chonburi-Ban Bueng Rd",
"postal_code": "20170", "country": "Thailand"},
{"city": "Tha Muang", "province": "Kanchanaburi", "street": "Sangchuto Rd", "postal_code":
"71110", "country": "Thailand"},
{"city": "Nang Rong", "province": "Buri Ram", "street": "Chokchai-Dej Udom Rd",
"postal_code": "31110", "country": "Thailand"},
{"city": "Phon", "province": "Khon Kaen", "street": "Phon-Chum Phae Rd", "postal_code":
"40120", "country": "Thailand"},
{"city": "Pak Chong", "province": "Nakhon Ratchasima", "street": "Thanarat Rd",
"postal_code": "30130", "country": "Thailand"},
{"city": "Sattahip", "province": "Chonburi", "street": "Sattahip-Nong Nooch Rd",
"postal_code": "20180", "country": "Thailand"},
{"city": "Ban Pong", "province": "Ratchaburi", "street": "Ban Pong Rd", "postal_code":
"70110", "country": "Thailand"},
{"city": "Phanom Sarakham", "province": "Chachoengsao", "street": "Sarakham Rd",
"postal_code": "24120", "country": "Thailand"},
{"city": "Khlong Luang", "province": "Pathum Thani", "street": "Khlong Luang Rd",
"postal_code": "12120", "country": "Thailand"},
{"city": "Khao Kho", "province": "Phetchabun", "street": "Khao Kho Viewpoint Rd",
"postal_code": "67270", "country": "Thailand"}
]

```

Generate realistic house numbers

```
def generate_house_number():
```

```
    return str(random.randint(1, 999)) + random.choice(["", "/1", "/2", "/A", "/B"])
```

Generate address rows for SQL

```
def generate_address_rows(num_customers=300, max_addresses_per_customer=3):
```

```
    rows = []
```

```
    address_id = 1
```

```
    for customer_id in range(1, num_customers + 1):
```

```
        num_addresses = random.randint(1, max_addresses_per_customer)
```

```
        for _ in range(num_addresses):
```

```
            addr = random.choice(thai_addresses)
```

```
            house_number = generate_house_number()
```

```
            row = f'({address_id}, {customer_id}, {house_number}, {addr['street']},
'{addr['city']}', '{addr['province']}', '{addr['postal_code']}', '{addr['country']}')'
```

```
        rows.append(row)
        address_id += 1
    return rows
# Generate SQL INSERT statement
def generate_sql_insert():
    rows = generate_address_rows()
    sql = "INSERT INTO address (address_ID, customer_ID, house_number, street, city,
province, postal_code, country) VALUES\n"
    sql += ",\n".join(rows) + ";"
    return sql

# Output the SQL
print(generate_sql_insert())
```

References

- Supermarket / Superstore Dataset Bundle, Kaggle [Supermarket / Superstore Dataset Bundle](#)
- Tangworakitthaworn, Preecha. CSS325 Project 1 Assignment Brief (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 3: Data Modeling Part 1 Business Requirement Analysis and Data Model (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 4: ER Diagram (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 5: EER Diagram (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 6,7: Relational Data Model and Mapping ERD to Relational Database Schema (PDF provided).
- Tangworakitthaworn, Preecha. Lecture 10: SQLPart1 DDL DML (PDF provided).